

Selective Knowledge Pinning for LLM Agent Teams: Traceable, Observable, and Cheaper Shared Context

Abstract—LLM agent teams share more than artifacts: a finding, a design decision, or a constraint must reach the agents that depend on it, and when it changes, the work derived from it must be redone. In practice this knowledge is pasted into prompts, so nothing records which decision used which version, a change either re-runs everything or relies on someone’s routing table, and the collaboration cannot be observed without reading transcripts. We extend a transformation-based agent runtime, whose LLM calls are bounded by declared footprints and version stamps, with three mechanisms for *knowledge*. (1) A versioned, access-controlled *shared context* whose *selected* items are pinned into each binding’s stamp, and which fails closed when a declared item cannot be resolved. (2) An *influence query* over the recorded pins. (3) An *observability plane*: a typed event timeline, optionally projected to Nostr relays as signed events, designed not to influence the run. We state the properties the design gives and the assumptions they need, and test them on **XX** DevBench repositories with **XX** open-weight model(s) in **XX** runs of a five-revision schedule against four distribution policies: naive paste, hand-routed paste, whole-context pinning, and item-level pinning. Item-level pinning delivered revised knowledge to **XX** of the artifacts that had to change (routed paste: **XX**), at **XX**× fewer tokens than naive paste and **XX**× the cost of routed paste, which a team must maintain by hand. Attaching Nostr left state and prompts identical in **XX** of replays, and **XX** of **XX** injected faults were surfaced; two probes of what the design cannot see stayed silent, as expected.

Index Terms—LLM-based multi-agent systems, shared context, traceability, observability, model transformation, incremental computation, software engineering agents

I. INTRODUCTION

Teams of LLM agents now analyze, design, implement, and test software together. Role-based frameworks assign an analyst, a coder, and a tester to separate model instances [1]. Others encode standard operating procedures as hand-offs between roles [2], [3], or coordinate agents around repository issues [4]. He et al. review this landscape in TOSEM and list, among its open problems, how agents share what they know and how their collaboration is organized [5]. The failure data point the same way. Cemri et al. identify *inter-agent misalignment*, where an agent withholds information, ignores another agent’s input, or acts on a stale view of it, as one of three failure categories of multi-agent LLM systems [6].

What a team transfers. Software engineering has studied knowledge transfer in *human* teams for two decades, and we use those studies as an organizing frame for agent teams; we do not claim a complete taxonomy. Ko et al. observed 17 developers. They most often sought awareness about artifacts

TABLE I
ORGANIZING FRAME: KNOWLEDGE A TEAM TRANSFERS, WHAT HUMAN-TEAM STUDIES OBSERVED, THE CHANNEL LLM AGENT TEAMS USE TODAY, AND THE ISSUE THAT CHANNEL RAISES.

Kind	Human teams	Channel in LLM teams	Issue
KT1 artifact content	passed between stages	role hand-off, SOP documents [1]–[3]	I1 loss and distortion in transfer [6]
KT2 cross-cutting knowledge	design knowledge most often deferred [7]	text pasted into prompts or context files [12]; experience pools [13]	I2 influence is untraceable
KT3 change and dependency	unmet coordination needs associated with failures [9]	re-run everything, or a planner [14]	I3 stale knowledge; I4 redundant regeneration
KT4 awareness	most often sought [7]; distance delays work [10]	transcripts and trajectories [15], [16]	I5 opaque collaboration

and coworkers, and the questions they most often had to defer concerned design and intended behavior, whose only source was an unavailable colleague [7]. De Souza and Redmiles describe how developers work to find out who depends on a change and whose changes affect them [8]. Cataldo and Herbsleb associate coordination requirements that a team leaves unmet with lower productivity and more failures [9]. Herbsleb and Mockus find that work items spanning sites take about two and a half times as long as co-located ones, and trace the delay to the larger number of people involved and to weaker cross-site communication [10]. From these studies we take four kinds of knowledge that a team transfers (Table I). **KT1** is the content of *artifacts* passed between stages. **KT2** is *cross-cutting knowledge* that is not an artifact of any stage: a design decision, a constraint, a review finding. **KT3** is knowledge of *change and dependency*: what changed, and who must react. **KT4** is *awareness*: who did what, and who knows what [11]. Questions about runtime behavior, such as the cause of a program state [7], fall outside this frame.

Issues raised in LLM agent teams. **I1**: in a transfer, content is lost or distorted, which is the inter-agent misalignment of Cemri et al. [6]. Missing knowledge also hurts a single agent: Zhang et al. trace part of the hallucinations in repository-level code generation to conflicts with project context the model was not given [17]. **I2**: once knowledge is pasted into a prompt, nothing records which decision used which version of it. Liang et al. argue from interviews that prompts are programs and call for program-like tooling [12]. Even links that humans

must record by hand are often missing: Rath et al. found only about 60% of commits in six open-source projects linked to an issue [18]. **I3**: when knowledge changes, the work derived from it goes stale unless someone knows what depends on it, the coordination problem of human teams [8], [9]. For code, CodePlan needs dependency and change may-impact analysis to propagate one edit through a repository [14]. **I4**: to avoid I3, a team re-runs more than it must, and every repeated call re-reads its context. Token limits are among the challenges LLM application developers report [19], long inputs are used less reliably [20], and cost now counts alongside effectiveness in evaluations of SE agents [21]. **I5**: agents leave transcripts, and explaining why an agent run succeeds or fails still takes manual trajectory analysis [15]. Protocols such as MCP and A2A standardize how messages travel [22]–[24], but they do not record which *knowledge version* an agent used. We know of no study that measures I3 or I4 in agent teams; they follow from the channel, and RQ1 and RQ2 below provide data on them.

Gap. Current frameworks pass knowledge as messages. MetaGPT, for instance, lets roles subscribe to a shared message pool [2], and conversational frameworks pass it in dialogue [3], [25]. They do not record which version of a shared item a generated value was conditioned on, and they do not decide which values a change to that item invalidates. Earlier work on the runtime we extend addresses KT1. There, each hand-off is a model-to-model (M2M) transformation: a deterministic engine owns structure, trace links, and obligations, and the LLM fills attribute values only through validated *stochastic bindings* that read a declared source footprint [26]. KT2–KT4 knowledge stays outside that engine. A security reviewer’s finding that `download()` must not write outside its target directory is not an element of any view, so the engine cannot tell which values used it (I2), which must be redone when it changes (I3, I4), or who used it (I5).

Approach. We add a collaboration plane beside the transformation plane. Knowledge becomes typed, versioned, attributed *context items* under an access policy. A binding declares, in its rule, which items it reads, and the digest of exactly those items joins its version stamp. The invalidation rule is deliberately the standard one of build systems, a hash of a declared read set [27]. What is new is what it is applied to and what comes with it. The input is knowledge that is not an artifact; it is access-controlled and resolved for the agent that owns the target view; and it fails closed when it cannot be resolved. The leaves are stochastic, so a stamp caches *a* sample, not *the* value. Each obligation records whether its cause was the source or the context. The *pins* recorded at acceptance make it a query to find every value generated with an item in its prompt; we call this set the item’s *influence*, meaning exposure, not causal effect (I2). Every transition is emitted as a typed event through one choke point. The timeline can be projected to Nostr relays as events signed with each agent’s own key, so a run that spans machines can be attributed without a trusted central server (I5). Compared with the earlier runtime, the

delta is the context store, item selection in the stamp, cause recording, pins and the influence query, the event plane, and relay sync. The declaration of which items a binding reads is still written by hand. The gain over a hand-kept routing table is that the declaration sits next to its consumer, the engine enforces it, and its use is recorded. Three limits matter. The design cannot see knowledge pasted into a prompt literal. It does not judge whether knowledge is right. And it addresses I1 only by giving the model labelled, attributed items in place of free text. We test the first two limits.

Research questions. **RQ1 (delivery and traceability)**: when shared knowledge is revised, does the revised knowledge reach the artifacts that depend on it, and does the recorded influence match what the agents were shown? **RQ2 (cost)**: what does item-level pinning cost against naive paste, hand-routed paste, and whole-context pinning, and does it preserve the retention of stated constants? **RQ3 (invariants and failure surfacing)**: does attaching the observability plane leave the run unchanged, how completely do events reconstruct it, what does it cost, and which injected failures are surfaced and which are not?

Contributions. **(C1)** Shared knowledge as versioned, access-controlled context whose *selected item digest* joins the binding stamp. It is specified in five algorithms over the running example of Section II, with three properties (selective invalidation, fail-closed resolution, non-interference), each stated with its assumptions and the cases it does not cover (Section III). **(C2)** An influence query over pinned trace links, and an observability plane with a privacy-tiered event schema and a signed Nostr projection whose receiver checks seven conditions before it applies a context version. **(C3)** An empirical study on **XX** DevBench repositories that compares item-level pinning with naive paste, hand-routed paste, and whole-context pinning, tests non-interference by replay, and injects 21 faults, including two probes of what the design cannot detect (Sections IV–V). The harness, the logs, the reference-validity script, the script behind every number of the running example, and the script that generates every number of the study are released (Section IX).

II. BACKGROUND AND RUNNING EXAMPLE

We introduce the runtime we extend on one running example, which every algorithm of Section III uses again. A released script runs the example with a scripted LLM and reproduces every count quoted in Sections II and III.

A. Running example: the *chakin* team

chakin is a DevBench repository [28]: a Python library that downloads pre-trained word vectors. Its UML class diagram has three public functions, `load_datasets()`, `download(number, name, save_dir)`, and `search(lang)`, and one test method, `test_download_by_name`, which the team treats as an operation like the others. The study adds five *canary* operations. Each canary’s requirement states a constant, for example a threshold, that the generated code must keep; canaries measure whether hand-offs preserve stated facts.

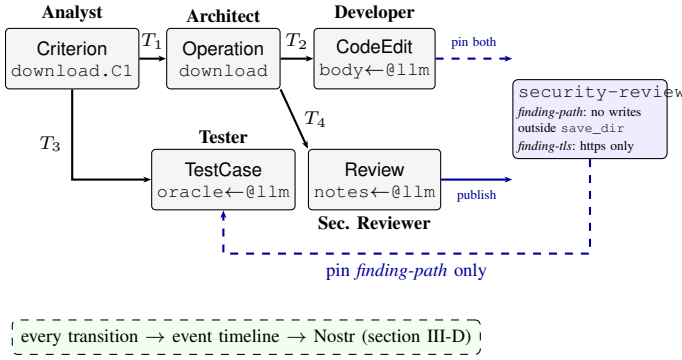


Fig. 1. Running example on DevBench chakin, shown for download. Solid arrows: hand-offs, the transformation plane (Section II). Blue: the collaboration plane, a shared context and its pins (Section III). Green: the observability plane.

That makes nine operations. The team has five agents (Figure 1). The *Analyst* owns view M_{req} , with one user story and one acceptance criterion per operation, for example download.C1. The *Architect* owns M_{arch} , with one Operation per story. The *Developer* owns M_{code} , with one CodeEdit per operation whose body is a function. The *Tester* owns M_{test} , with one TestCase per criterion whose oracle is a test function. The *Security Reviewer* owns M_{sec} , with one Review per operation, and is added after the first run. Four hand-offs connect the views: T_1 Req2Arch, T_2 Arch2Code, T_3 Req2Test, which derives tests from the requirements rather than from the design, and T_4 Arch2Sec.

B. Hand-offs as transformations

Definition 1 (Team, hand-off, trace link). A team is a megamodel, a model whose elements are models and the transformations between them: $\mathcal{T} = (A, V, \mathbf{T}, \omega, \varphi)$ with agents A , view metamodels V , hand-off transformations $\mathbf{T}$, write rights $\omega : V \rightarrow A$ (one owner per view), and an acceptance predicate φ [26]. A hand-off T_{ij} is a set of ATL-style rules [29] that read M_i and write M_j . A rule r has a source pattern p_r , a guard g_r , and target bindings. For each match m it creates a target t and records a persistent trace link (m, t, r) in the trace model TL [30], [31]. We write $\omega(t)$ for the owner of t 's view.

Target bindings come in two kinds. A *structural* binding is an OCL-like expression the engine evaluates, such as `name <- op.name`. A *stochastic* binding `f <- @llm(prompt, e)` delegates to the LLM a value that no expression can compute. Listing 1 shows T_2 . Its footprint `op.codeContext()` is the operation's signature and its specification, which T_1 copies structurally from the criterion.

Definition 2 (Footprint, stamp, obligation). The expression e of a stochastic binding b defines its *footprint* $fp_b(m)$, the only source values the LLM may see. Let H be SHA-256 truncated to 16 hexadecimal digits (64 bits), computed over a canonical encoding of the footprint's attribute values. When a value is accepted, the trace link stores the stamp $stamp(t, b) = H(fp_b(m))$. The pair (t, b) is an *obligation* iff a stamp exists

Listing 1. Hand-off T_2 (Architect $\rightarrow$ Developer) of the running example, abridged.

```
rule Operation2CodeEdit {
  from op : Arch!Operation
  to ce : Code!CodeEdit (
    name      <- op.name,           -- structural
    operation <- op,                 -- structural
    body      <- @llm('Implement this operation ...',
                      op.codeContext()), -- stochastic
    @check body.compiles()
    and body.definesName(op.name) ) }
```

Algorithm 1 Executing a hand-off [26]

```
1: function HANDOFF( $T_{ij}, M_i, M_j, TL$ )
2:   match  $M_i$ ; create, link, or delete targets; evaluate structural
   bindings                                     ▷ structure
3:   for  $(m, t, r) \in TL$ , stochastic  $b$  of  $r$  do
4:      $d \leftarrow H(fp_b(m))$ 
5:     if  $stamp(t, b) = d$  then continue           ▷ fresh: no call
6:     if  $failed(t, b) = d$  then report escalation; continue
7:      $\pi_0 \leftarrow prompt_b \oplus render(fp_b(m)); \pi \leftarrow \pi_0$ 
8:     for  $a = 1..k$  do
9:        $v \sim LLM(\pi)$ 
10:      if  $chk_b(v)$  then  $t.fb \leftarrow v$ ;  $stamp(t, b) \leftarrow d$ ; return
11:       $\pi \leftarrow \pi_0 \oplus feedback(v)$            ▷ retry with the rejection
12:       $failed(t, b) \leftarrow d$  if a sample was rejected; escalate
```

and differs from H of the current footprint, and *new* iff no stamp exists. The predicate φ holds iff every match is covered, no obligation or escalation is open, and all stamps are fresh.

Algorithm 1 executes one hand-off. Line 2 owns all structure. Lines 3–12 call the LLM only for bindings whose footprint digest differs from their stamp, at most k times, and each sample must pass the binding's validator chk_b (the @check clause). A binding that exhausts its budget is *escalated*: it is reported to the team as an open problem, and $failed(t, b)$ remembers the footprint so that the same prompt is not retried until the footprint changes. Transport failures are not remembered. The runtime has two modes. In *engine mode* it calls an LLM backend itself. In *host mode* it hands each pending prompt to an external agent, such as a coding assistant, and judges the value that agent submits. Algorithm 1 is incremental computation [27], [32] with a stochastic function at the leaves.

Example. After the first run all nine body bindings are stamped. Suppose the Analyst edits criterion download.C1. T_1 copies the new text into the specification of download, so $fp_{body}(\text{download})$ changes, as do the footprints of the Architect's signature and of the Tester's oracle for download.C1. Exactly these three bindings are obliged, and three LLM calls are made; the eight other bodies stay fresh. KT1 transfer is thus already traceable and selective.

C. Knowledge that is not an artifact

Now the Security Reviewer reads download and publishes two findings (Figure 1). *finding-path*: save_dir comes from the caller, so the function must refuse to write outside it. *finding-tls*: vectors must be fetched over https only. Both are

TABLE II

BINDINGS RE-RUN PER CHANGE IN THE RUNNING EXAMPLE (9 BODIES, 9 ORACLES). *Declared*: CONSUMERS UNDER THE PINS OF FIGURE 1. *Semantic*: BINDINGS WHOSE VALUE THE FINDING IS ABOUT. PASTE RE-RUNS ON ANY CHANGE OF TEXT; CTX PINS THE WHOLE CONTEXT, CTXI THE ITEMS OF FIGURE 1.

Change	Semantic	Declared	paste	routed	ctx	ctxi
S1 revise <i>finding-path</i>	2	18	18	18	18	18
S2 revise <i>finding-tls</i>	1	9	18	9	18	9
S3 add unrelated note	0	0	18	0	18	0
S4 identical rewrite	0	0	0	0	0	0
Routing declared where?			–	table	rule	rule
Routing enforced and recorded?			no	no	yes	yes

KT2 knowledge. Neither is an element of M_{arch} , so neither enters fp_{body} , and Algorithm 1 cannot see them. Today a team distributes such findings in one of two ways.

Naive paste (paste). The text of every finding is pasted into every consumer’s prompt. The prompt literal is not part of the footprint, so a revision creates no obligation. The operator, who has no record of dependencies, must re-run every consumer (I4) or risk stale code (I3). Nothing records which body was generated under which version of *finding-path* (I2).

Routed paste (routed). Each finding is pasted only into the prompts that need it, and only those are re-run. Its cost is minimal, but its routing table (“*finding-tls* goes to the Developer only”) is a dependency graph that an operator keeps and applies by hand. Unmet coordination needs of this kind are associated with failures in human teams [9].

Table II states the problem on the example. **routed** matches the declared set only while its table is right and is applied. **paste** re-runs everything. Neither records influence. The approach should re-run what the declarations require, enforce and record them, keep a missing or revoked finding from being silently replaced, and make all of it observable without changing the run. The table also shows two limits. Both findings concern *download*, which is two bindings (*Semantic*), yet all nine bodies consume them, because a declaration is per binding, not per match. And no mechanism here can tell whether *finding-path* is correct. In the study (Section IV) the findings are synthetic markers, so that whether knowledge reached an artifact is decidable; S1 and S2 correspond to its revisions R1, R2, and R5, S3 to R3, and S4 to R4.

Why not shared memory, retrieval, subscription, or prompt versioning? A blackboard or memory store [33], [34] shares knowledge but does not say which outputs used which version. Retrieval-augmented generation selects knowledge per call without a declaration, but the selection then depends on the retriever, so a stamp over it would not be stable. Subscription to a message pool [2] declares interest, but delivery does not invalidate values generated earlier. Versioning prompts records which prompt version ran, but not which output was derived from which knowledge version, and it obliges nothing. The approach combines a declared read set, invalidation per binding, and a record per accepted value.

III. APPROACH: SHARED CONTEXT AS A COLLABORATION PLANE

Figure 1 shows the three planes, and Table IV, at the end of this section, maps each issue to its mechanism, its property, and what is not covered. Four design goals follow from Section II-C. **D1** (enforced declaration): the dependency is declared at the consumer and enforced by the engine, not decided by an operator at each change. **D2** (fail closed): a binding whose declared knowledge cannot be resolved is never answered with empty or invented knowledge. **D3** (non-interference): observing a run must not change it. **D4** (authority): only authorized agents read or write knowledge, and every item names its real author.

A. Context model and store

Definition 3 (Item, snapshot, policy). A *context item* is a record $i = (id, type, content, author, provenance)$. A *context* c is a sequence of immutable *snapshots* $\sigma_1, \sigma_2, \dots$, each a set of items. For a set of items S , $dg(S) = \text{SHA256}$ of the canonical records of S sorted by id. For a list of ids I , $\sigma|_I$ selects those items, and $\sigma|_*$ is all of them. A *policy* $\pi_c = (owner, W, R, vis, expiry)$ names writers W , readers R , a visibility (*private*: only listed agents read; *team*: every agent reads; *relay*: also synchronized across machines), and an optional expiry.

The digest covers the whole record and no timestamp. Identical items give an identical digest at any time. A change of authorship is a change, so re-attributing an item obliges its consumers, which is intended: the author is part of what the model is shown. *finding-path* has type *security-finding*, author *SecurityReviewer*, and as provenance the trace link of the **Review** from which it was derived.

Algorithm 2 is the only path by which an agent reads or writes a context, whatever its route (engine, command line, MCP tools, or relay). $\triangleleft$ replaces items by id, and G_c holds runtime read grants, which only the owner adds or revokes; readers declared in the team file change only when that file changes. Three operations bypass it, and none is exposed to agents: the engine creates declared contexts and loads their policy from the team file; relay sync appends a version after the checks of Algorithm 6; and the influence query reads snapshots on behalf of the workspace operator. *Example.* The team declares *security-review* with owner and writer *SecurityReviewer* and readers *Developer* and *Tester*. Its first snapshot σ_1 is empty. The Reviewer calls **WRITE** with $v_{exp}=1$ and both findings, which produces σ_2 with each item attributed to *SecurityReviewer*. If the Tester tries to add an item, line 3 raises **DENIED**. A second writer, if one were declared, could not submit an item attributed to the Reviewer (line 6). Two Reviewer sessions that both start from σ_2 cannot both write: the second gets **CONFLICT** at line 4 and must re-read. A rewrite with identical text (S4) returns the current snapshot at line 9, so no version is created and nothing is obliged. A context never creates model

Algorithm 2 Context store: authorized write, resolve, and grant

```

1: function WRITE( $c, a, v_{exp}, Add, Del$ )
2:    $\sigma \leftarrow$  current snapshot of  $c$   $\triangleright$  NOTFOUND if none
3:   if  $\neg \text{CANWRITE}(\pi_c, a)$  then raise DENIED
4:   if  $v_{exp} \neq \sigma.v$  then raise CONFLICT  $\triangleright$  no last-writer-wins
5:   for  $i \in Add$  do  $\triangleright$  D4: attribution
6:     if  $i$  names an author other than  $a$  then raise DENIED
7:      $i.author \leftarrow a; i.provenance.agent \leftarrow a$ 
8:    $S \leftarrow (\sigma \setminus Del) \triangleleft Add$   $\triangleright$  unknown id in  $Del$ : NOTFOUND
9:   if  $\text{dg}(S) = \text{dg}(\sigma)$  then return  $\sigma$   $\triangleright$  identical content: no version
10:  append and return  $\sigma' = (\sigma.v+1, S, \text{dg}(S), a)$ 
11: function RESOLVE( $c, a, I$ )
12:  if  $c$  unknown  $\vee \neg \text{CANREAD}(\pi_c, a)$  then raise UNAVAILABLE
13:   $\sigma \leftarrow$  current snapshot of  $c$ 
14:  if an id of  $I$  is not in  $\sigma$  then raise UNAVAILABLE
15:  return  $(\sigma, \sigma|_I)$ 
16: function GRANT( $c, a_o, a$ ) require  $a_o = \text{owner}(\pi_c); G_c \leftarrow G_c \cup \{a\}$ 
    CANWRITE: not expired  $\wedge a \in \{\text{owner}\} \cup W$ 
    CANREAD: not expired  $\wedge (a \in \{\text{owner}\} \cup W \cup R \cup G_c \vee \text{vis} \neq \text{private})$ 

```

Listing 2. Running example with pins: the Developer pins two items, the Tester one. Writing only 'security-review' pins the whole context ($I = *$).

```

body <- @11m('Implement ...', op.codeContext(),
  context = ['security-review#finding-path',
            'security-review#finding-tls']),
oracle <- @11m('Write a test oracle ...', c.text,
  context = ['security-review#finding-path']),

```

elements, grants write rights ω , or bypasses φ : knowledge informs values, and structure stays with the engine.

B. Pinning knowledge into the stamp

A stochastic binding may declare a sequence of context references $\rho_b = \langle (c_1, I_1), \dots, (c_n, I_n) \rangle$, where I_j lists item ids of c_j or is $*$ for all items (Listing 2). The two-argument form of Listing 1 is unchanged.

For a match m and the current snapshots, the selection is the sequence $sel_b = \langle (c_j, \text{dg}(\sigma_j|_{I_j})) \rangle_j$. The *effective footprint* and the stamp are

$$\begin{aligned}
fp_b^+(m) &= (fp_b(m), sel_b), \\
stamp_b(m) &= \begin{cases} H(fp_b(m)) & \text{if } \rho_b = \langle \rangle, \\ H(\tau, fp_b^+(m)) & \text{otherwise,} \end{cases} \quad (1)
\end{aligned}$$

where τ is a constant tag that keeps the two cases apart. A binding without references keeps exactly its earlier stamp, so existing workspaces stay fresh. From here on, Definition 2 reads with $stamp_b(m)$ in place of $H(fp_b(m))$, and φ also requires every declared reference to be resolvable.

Algorithm 3 differs from Algorithm 1 in six places. Line 2 authorizes the agent that *owns the target view*, not whoever runs the engine. Line 3 resolves the declared items. Line 4 fails closed. Line 5 computes the stamp from the selected items

Algorithm 3 A stochastic binding with pinned context (replaces lines 4–12 of Algorithm 1)

```

1: function PINNEDBINDING( $b, m, t$ )
2:   $a \leftarrow \omega(t); f \leftarrow fp_b(m)$   $\triangleright$  authorize the view owner
3:  try  $\mathcal{R} \leftarrow [\text{RESOLVE}(c, a, I) \mid (c, I) \in \rho_b]$ 
4:  catch UNAVAILABLE: emit context.error; block or escalate, not cached; return  $\triangleright$  D2
5:   $s_{src} \leftarrow H(f); s_{ctx} \leftarrow H(sel_b); s \leftarrow stamp_b(m)$ 
6:  if  $stamp(t, b) = s$  then return  $\triangleright$  fresh
7:  if  $failed(t, b) = s$  then report escalation; return
8:  if  $stamp(t, b) \neq \perp$  then  $\triangleright$  an obligation: record why
9:     $cause \leftarrow \{source \mid dep(t, b).src \neq s_{src}\} \cup \{context \mid dep(t, b).ctx \neq s_{ctx}\}$ 
10:  emit binding.stale, obligation.created(cause)
11:   $\pi_0 \leftarrow prompt_b \oplus render(f) \oplus LABELLED(\mathcal{R})$ 
12:  emit binding.requested, context.read, binding.prompt_prepared
13:  sample, validate, escalate as in Algorithm 1, lines 8–12; on accepting  $v$ :
14:     $t.fb \leftarrow v; stamp(t, b) \leftarrow s; dep(t, b) \leftarrow (s_{src}, s_{ctx})$ 
15:     $pins(t, b) \leftarrow [(c, \sigma.v, \text{dg}(\sigma), I, \text{dg}(\sigma|_I)) \mid (\sigma, \sigma|_I) \in \mathcal{R}]$ 
16:    emit binding.accepted(pins)

```

TABLE III

ALGORITHM 3 ON THE RUNNING EXAMPLE UNDER CTXI: THE STATE OF BODY(DOWNLOAD) AND OF ORACLE(DOWNLOAD.C1) AFTER EACH CHANGE, AND THE LLM CALLS OVER ALL 18 BINDINGS PLUS THE ARCHITECT’S SIGNATURES. O: OBLIGATION, WITH ITS RECORDED CAUSE.

Change	σ	body	oracle	Calls
publish, install Listing 2	σ_2	O (context)	O (context)	18
S1 revise <i>finding-path</i>	σ_3	O (context)	O (context)	18
S2 revise <i>finding-tls</i>	σ_4	O (context)	fresh	9
S3 add unrelated note	σ_5	fresh	fresh	0
S4 identical rewrite	σ_5	fresh	fresh	0
Analyst edits download.C1	σ_5	O (source)	O (source)	3
Tester dropped as reader	σ_5	fresh	blocked	0

only. Lines 8–10 record the *cause* of an obligation; for a value accepted before the binding had references, $dep(t, b)$ is taken as $(stamp(t, b), \perp)$, so installing the references reports cause *context*. Lines 11 and 15 render a labelled, read-only prompt section and store the pins; the section names, for every item, its context, version, snapshot digest, id, type, and author. Both come from the same resolved list $\mathcal{R}$.

Example. Table III runs Algorithm 3 through the changes. When the rules of Listing 2 are installed over values accepted without context, every body and oracle is obliged once, with cause *context*. After S1 both selections differ from their pinned digests: 18 obligations and 18 calls. After S2 only the Developer’s selection changed, so 9 calls are made and the oracles stay fresh. S3 adds an item that neither selection names, so no stamp changes. S4 creates no version. The Analyst’s edit of download.C1 changes s_{src} of three bindings (signature, body, oracle), and their events say *source*. When the team policy drops the Tester as a reader, line 3 raises for all nine oracles, already fresh or not. They are blocked (host mode) or escalated (engine mode), the event names the reason (agent ‘Tester’ may

not read context 'security-review'), and φ is false until access returns. The accepted oracles stay in the model. After S1 the trace link of `body(download)` holds the stamp, the dependency pair, and the pin (`security-review, 3, dg( $\sigma_3$ ),  $\langle path, tls \rangle, dg(\sigma_3 | \langle path, tls \rangle)$`).

Property 1 (Selective invalidation). Assume H has no collision on the inputs that occur (non-adversarial use), the rule of b (its footprint expression and ρ_b) is unchanged since acceptance, and every reference in ρ_b is resolvable. Then a binding accepted on match m is an obligation iff the footprint digest changed or the digest of the selected items of some $(c, I) \in \rho_b$ differs from its pinned value. It is re-sampled unless $failed(t, b)$ equals the new stamp. A new version of c whose selected items are unchanged, a revision of items outside I , and a change to a context not in ρ_b leave it fresh. The stamp does *not* cover the prompt template, the validator, the model, or the sampling settings. Changing ρ_b itself (adding, removing, or reordering references) obliges the binding, with cause *context*.

Argument. With the rule fixed, stamp equality is equality of H over fp_b^+ , and sel_b reads only the selected items. With 64-bit stamps and 10^2 – 10^3 bindings per workspace, the probability that some binding's new stamp collides with its old one over the study's revisions is below 10^{-10} . A writer who wants a collision can find one with about 2^{32} work, so the property assumes authorized writers do not attack the stamp. $\square$

Property 1 concerns the *declared* dependency. Relative to the declared selection there is no over-invalidation and no stale reuse of a declared item; as Table II shows, the declaration itself is per binding, not per match. Knowledge pasted into a prompt literal, or an edit of the prompt text, is invisible to the stamp. That is why pasted knowledge is untraceable (I2), and we test it as a limit (Section IV).

Property 2 (Fail-closed). If a reference in ρ_b is missing, expired, or not readable by $\omega(t)$, then b is not sampled, no stamp is written, and φ is false. The binding is blocked (host mode) or escalated without caching (engine mode); it is never answered with empty or invented context.

Argument. RESOLVE raises before line 11 builds a prompt, and the handler at line 4 only emits an event and returns. We check this on the code paths that build prompts and stamps: engine sampling, host submission, the freshness check behind φ , and acceptance. Two consequences are part of the design. A value accepted earlier stays in the model and remains readable downstream. And because expiry and revocation depend on policy and the clock, φ is not a function of model state alone, so one expired context blocks every dependent binding. $\square$

C. Influence queries

Given pins on trace links, $Infl(c, i) = \{(t, b) \mid \exists p \in pins(t, b) : p.c = c \wedge INC(p, i)\}$ is the set of accepted values currently in the trace that were generated with item i in their prompt (Algorithm 4). Each result carries its agent, pinned version, and a flag that says whether the pinned selection still

Algorithm 4 Influence (exposure) of a context item over pinned trace links

```

1: function INFLUENCE( $c, i, transitive$ )  $\triangleright i = \perp$ : any item of  $c$ 
2:    $D \leftarrow \{(t, b, \omega(t), p.v, dg(\sigma_{now}|_{p.I}) = p.dg_I) \mid p \in$ 
      $pins(t, b), p.c = c, INC(p, i)\}$ 
3:   if  $transitive$  then  $D \leftarrow D \cup DOWNSTREAM_{TL}(\{t \mid (t, \cdot) \in$ 
      $D\})$ 
4:   return  $D$ 
   INC( $p, i$ ):  $i = \perp$ , or  $i \in p.I$ , or  $p.I = *$  and  $i \in \sigma_{p.v}$ 

```

Algorithm 5 The single emission point

```

1: function EMIT( $type, corr, P$ )
2:   try require  $type$  is one of the 47 event types
3:     publish ( $type, seq, time, corr, REDACT(P, tier)$ ) to every
       sink
4:   catch any exception: count it, return  $\triangleright D3$ : never raises
5: function REDACT( $P, tier$ )
6:   content fields (prompt, value, reason, footprint, context con-
     tent)  $\mapsto (H(v), |v|)$  unless  $tier$  is debug and the field's flag is
       set
7:   in minimal: also drop lengths and metadata (pins, item ids,
       tokens)

```

equals the current one (false if an item was removed). DOWNSTREAM follows trace links forward from the elements found, which over-approximates exposure along recorded links, not along the data flow of footprints. *Example.* After the Analyst's edit, INFLUENCE(`security-review, finding-path`) returns 18 values: eight bodies pinned at version 4 and the body of download at version 5 (it was re-sampled after the edit), and likewise eight oracles at version 3 and one at version 5. Under *paste* there is no pin to query, because the finding text sits inside a rule's prompt literal; a text search over rule files recovers the same set only for someone who knows to look. Pins and prompt section come from the same $\mathcal{R}$, so in engine mode *Infl* equals "the items in the prompt" by construction. In host mode the equality also needs the host to return the stamp of the prompt it answered, which the runtime then checks; a value submitted without it is pinned to the snapshots current at submission. RQ1 therefore checks the implementation; it does not discover anything new about influence.

D. Observability plane

The engine reports every transition through EMIT (Algorithm 5). The 47 event types cover the team and hand-off lifecycle, target creation, binding requests, acceptances, rejections, and escalations, obligations, context writes, reads, and invalidations, and agent presence. Each event carries correlation ids: run, agent, hand-off, rule, target, binding, trace link, and contexts. Three privacy tiers decide what a payload contains: *minimal* (ids, digests, counts), *standard*, the default (also pins, versions, and token counts), and *debug* (raw content, field by field, behind explicit flags). *Example.* Under the standard tier, S1 produces this timeline for `body(download)`: `context.updated` (agent SecurityReviewer, version 3, digest), `binding.stale` and `obligation.created` (cause *context*), `binding.requested` (pins),

`context.read` (version 3, items *finding-path* and *finding-tls*), `binding.prompt_prepared` (prompt digest and length, not the prompt), and `binding.accepted` (pins, attempts, input and output tokens). From these events alone an observer can tell that the Developer re-implemented download because of version 3 of the review, and what it cost, without reading a prompt. A digest proves *which* prompt or context version an agent received; anyone who can guess a short value can confirm the guess from its digest, and target keys and error messages remain visible. “No content” therefore means no raw prompts, values, or finding text.

Property 3 (Non-interference). Assume sinks terminate and do not mutate the data they are given, no wall-clock time enters engine decisions, and no context expires during the run. Then for fixed LLM answers the final model state, trace links, stamps, and the multiset of prompts are independent of which sinks are attached, of their failure, and of relay availability. Context sync (Algorithm 6) writes snapshots into the store, so it is a separate channel and is not covered.

Argument. EMIT catches every ordinary exception and returns, and no engine decision reads its result or a sink. The Nostr sink appends each signed event to a durable outbox before the first delivery attempt, so a failing relay delays observation, not the run. Sinks run synchronously on the engine’s thread, so a slow sink delays the run; the expiry assumption is needed because such a delay could otherwise push a resolution past an expiry. This is a design invariant tested by replay (Section IV), not a theorem about the implementation. $\square$

E. Projection to Nostr and cross-machine sync

Nostr is a minimal protocol for signed events [35]. An event is a JSON object whose id is the hash of its content and metadata and which carries a BIP-340 Schnorr signature by its author’s public key [36]. Relays store and forward events without trusting or being trusted by anyone; readers verify. A *regular* kind keeps every event, and an *addressable* kind keeps only the latest event per author and key. We project timeline events to a regular kind and agent presence to an addressable one. A context with *vis* = *relay* is also *synchronized*: each new version is published as a snapshot event signed with the writing agent’s own key, never with an engine key. Algorithm 6 decides whether a receiving workspace applies it; checks C1–C4 and C6 run when the event is decoded.

The local store stays the database and the local policy the authority (C4): a relay can deliver knowledge, but never grant the right to write it. *Example.* The Security Reviewer works on one machine and the Developer on another. The Reviewer’s S1 write creates σ_3 , which is published signed with the Reviewer’s key. The Developer’s workspace checks C1–C7 and appends σ_3 , and from then on Algorithm 3 on that machine obliges the same 18 bindings as in a single workspace. An event that claims to come from the Reviewer but is signed with the Tester’s key fails C3. An event the Tester signs as itself fails C4, because the local policy does not list the Tester as a writer. A σ_5 that arrives before σ_4 waits; when a pull ends with

Algorithm 6 Applying a received context snapshot (untrusted input)

```

1: function RECEIVE( $e$ )  $\triangleright e$  carries  $c, \sigma$ , writer  $a$ , base digest  $\beta$ 
2:   C1: NIP-01 id and BIP-340 signature are valid
3:   C2: schema, kind, and team namespace match
4:   C3:  $e$  is signed with the registered key of  $a$ 
5:   C4:  $c$  exists locally and CANWRITE( $\pi_c^{local}, a$ )
6:   C6:  $\text{dg}(\sigma.items) = \sigma.dg$   $\triangleright$  content integrity
7:   on failure emit context.rejected(reason) and return
8:   add  $e$  to Pending; apply pending events in version order until
   none applies
9: function APPLY( $c, \sigma, a, \beta$ )  $\triangleright$  C5, C7
10:   $\kappa \leftarrow$  current snapshot of  $c$ 
11:  if  $\sigma.v \leq \kappa.v$  then return duplicate if  $\sigma_{\sigma.v}$  has digest  $\sigma.dg$ ,
   else old
12:  if  $\sigma.v > \kappa.v+1$  then return pending  $\triangleright$  wait for
   predecessors
13:  if  $\beta \neq \text{dg}(\kappa)$  then return conflict  $\triangleright$  C5: other base
14:  if some new or changed item of  $\sigma$  is not authored by  $a$  then
   return rejected  $\triangleright$  C7
15:  append  $\sigma$ ; emit context.received; return applied

```

TABLE IV
ISSUES (TABLE I), MECHANISMS, AND WHAT IS AND IS NOT GUARANTEED.

Issue	Mechanism	Guarantee (assumption)	Not covered
I1	labelled, attributed items (Alg. 2, 3)	the prompt contains exactly the declared items, labelled	whether the model uses them; whether they are right
I2	pins, INFLUENCE (Alg. 4)	<i>Infl</i> equals items in the prompt (engine mode, by construction)	causal influence; footprint data flow
I3	selected digest in stamp (Alg. 3)	Property 1 (rule fixed, no collision)	knowledge in prompt literals; edits of prompt, model, validator
I4	item-level selection	re-run exactly the declared consumers	per-match relevance (9 bodies, 2 needed)
I5	EMIT, Nostr (Alg. 5, 6)	Property 3 (sinks terminate, no expiry)	network timing; concurrent writers
D2, D4	RESOLVE, WRITE, C1–C7	Property 2; attribution	liveness: an expired context blocks its dependents

events still waiting, they are reported as conflicts. Duplicates are recognized by version and digest. Two valid events for the same version from the same base form a fork: the first applied wins and the second is reported as old. This is weaker than the store’s optimistic concurrency, because which event wins depends on delivery order. The protocol therefore assumes a single writer per context and gives no convergence guarantee for concurrent writers, the trade-off against replicated data types [37].

IV. STUDY DESIGN

A. Subjects, models, and teams

Repositories. We use DevBench’s Python repositories [28], restricted by a mechanical rule: a repository qualifies only if its *reference* implementation passes its own unit and acceptance tests in our environment (a released script and its output). **XX** of **XX** qualify; of the others, one needs a download that fails offline and the other fails **XX** of its own unit tests. Each repository yields a team of Analyst, Architect, Developer, and Tester with one user story per UML operation

TABLE V
COVERAGE OF THE CONTEXT STUDY. RUNS THAT DID NOT COMPLETE
ARE REPORTED, NOT DROPPED.

Model	Repositories	Seeds	Runs (repo×policy×seed)
qwen2.5-coder:7b	XX	XX	XX
qwen3.8:27b	XX	XX	XX

(4 to 17 per repository) and five canary operations, plus a Security Reviewer who owns the shared context. **Models.** qwen2.5-coder:7b and qwen3.8:27b, served locally by Ollama (versions and model digests are recorded in the artifact), temperature 0.2, a 4096-token output cap, thinking disabled to bound output, per-call token counts reported by the server, $k=2$, and the sampling seed passed to the server. **Scope.** The context study covers **XX** repositories and **XX** seed(s) across **XX** model(s); coverage per model is in Table V.

B. Knowledge, policies, and the revision schedule

The context holds two findings with unguessable markers. *finding-code*: every implementation must contain the comment `# SEC-C-<tag>`. *finding-test*: every test oracle must contain `# SEC-T-<tag>`. The Developer’s `body` bindings consume the first, the Tester’s `oracle` bindings the second. Tags derive from (repository, seed, finding, version), so whether knowledge reached an artifact is a string test and a stale marker is distinguishable from a fresh one. The findings are synthetic and instruction-like on purpose: ground truth must be decidable. They test whether knowledge is *delivered and kept current*, not whether it is good advice (Section VII). Four *knowledge-distribution policies* are compared under identical model, rules, footprints, temperature, and resample budget:

- **paste**: the text of *every* item of the context is pasted into every consumer’s prompt; on any text change the operator, who has no routing information, re-runs every consumer.
- **outed**: each finding is pasted only into the prompts of the consumers that need it, and only those are re-run when it changes. This is the strongest manual practice, a hand-maintained routing table kept correct by assumption.
- **ctx**: shared context, each binding pins the *whole* context.
- **ctxi**: shared context, the Developer pins *finding-code* and the Tester *finding-test* only.

After the build (R0), five revisions each have a known ground-truth set of artifacts that must change: R1 revises *finding-test* (all oracles); R2 revises *finding-code* (all bodies); R3 adds an unrelated style note (none); R4 rewrites a finding with identical content (none); R5 revises *finding-code* again (all bodies). R3 is the case that distinguishes selective from whole-context pinning and tests Property 1; R4 tests only the store’s de-duplication of identical content. Bindings that never obtained an accepted value hold no stamp and are retried on every run in every arm; they are failures of the model, not obligations.

C. Measures

RQ1. Delivered: the share of ground-truth artifacts carrying the new marker after the step; *stale left*: the share still carrying the old one; *regeneration churn*: values whose text changed outside the ground truth, per step (largely sampling noise, reported as the cost of unneeded regeneration, not as a quality loss). As *implementation checks* of Property 1 and of *Infl*: *impact precision and recall* of the bindings the runtime marks as obligations against the ground truth (for the paste arms, the consumers the operator re-runs), the *spurious re-run share* on no-change steps (R3, R4), and *influence precision/recall* of *Infl* against the context items present in the last prompt prepared for each accepted binding, read back from prompt digests in the timeline and the recorded prompts. For the context arms these must be near 1 if the implementation matches the design; for the paste arms they are determined by the policy’s definition, so we report them descriptively and do not test them. **RQ2.** Tokens and LLM calls for the five revisions and for the build, per policy; the *fidelity guard*: canary retention and unit-test pass of the generated module after the build and after the last revision, with non-inferiority margin of one canary of five per repository. **RQ3.** (a) *Re-construction*: the share of accepted values whose footprint version and pinned context versions are recovered from the timeline events alone and from the verified relay events alone, against the engine’s trace links; because the engine emits the events, this checks that the schema is *sufficient*, not that it is independent evidence. (b) *Projection control*: the `ctxi` run is *replayed* with every prompt answered from its recording under four conditions (no sink, timeline, Nostr, Nostr with relay down); final state hash and prompt multisets must be identical, and a prompt absent from the recording counts as a violation. (c) *Overhead* per event of the timeline and of signing and queueing, measured with the LLM answered instantly. (d) *Privacy*: occurrences of markers, code, or prompt text in standard-mode timeline and Nostr timeline events. (e) *Fault injection*: 21 injections (Table IX): 18 faults in four families (context access, LLM, Nostr integrity, Nostr availability), one benign control (an unrelated context changes), and two *design-limit probes*, namely an edit to the rule’s prompt text (an undeclared dependency) and a contradictory finding published by an authorized writer. A fault is *surfaced* if the intended signal appears and *silent* if state is corrupted or the run reports acceptance while it is active. The faults and their intended signals were chosen by us, so this is a verification suite for the design, not an independent estimate of detection rates; the probes are included to show where the answer is no.

D. Analysis and procedure

The unit of analysis is a repository for a model, averaged over seeds, because units sharing a repository are not independent; per-repository values are released and shown in the figures. Paired differences between policies use a 95% bootstrap interval (10,000 resamples) and an exact two-sided sign-flip permutation test (≤ 16 units); cost ratios use summed cost. We apply no significance tests to quantities that the

TABLE VI

REPLICATION OF THE EARLIER CHANGE-PROPAGATION EVALUATION ON EVERY REPOSITORY THAT PASSES THE REFERENCE CHECK, MEAN OVER SEEDS. F: FREE TEXT, S: SHARED SCHEMA, A: TRANSFORMATION-BASED HAND-OFFS.

Metric	qwen2.5-coder:7b			qwen3.8:27b		
	F	S	A	F	S	A
Canaries kept	XX	XX	XX	XX	XX	XX
Build tokens (k)	XX	XX	XX	XX	XX	XX
Impact precision	XX	XX	XX	XX	XX	XX
Impact recall	XX	XX	XX	XX	XX	XX
Tokens per change (k)	XX	XX	XX	XX	XX	XX

TABLE VII

RQ1. REVISION STEPS R1–R5 POOLED OVER REPOSITORIES, SEEDS, AND MODELS. DELIVERED/STALE: SHARE OF GROUND-TRUTH ARTIFACTS CARRYING THE NEW/OLD MARKER. CHURN: VALUES CHANGED OUTSIDE THE GROUND TRUTH PER STEP. IMPACT P/R, EXACT, SPURIOUS AND INFLUENCE P/R ARE IMPLEMENTATION CHECKS (SECTION IV); FOR PASTE AND ROUTED THEY FOLLOW FROM THE POLICY.

Policy	Delivered	Stale	Churn	Impact P	Impact R	Exact	Spurious	Infl. P/R
Paste	XX	XX	XX	XX	XX	XX	XX	–
Routed	XX	XX	XX	XX	XX	XX	XX	–
Ctx	XX	XX	XX	XX	XX	XX	XX	XX
Ctx-item	XX	XX	XX	XX	XX	XX	XX	XX

policy definitions fix (paste’s impact precision, spurious re-runs, fault surfacing), which we report descriptively. Effect directions were fixed before the final runs but not registered externally: **H1** ctxi’s delivery is not lower than paste’s or routed’s; **H2** ctxi costs less than paste and ctx and about as much as routed, with canary retention non-inferior; **H3** Nostr changes neither state nor prompts, and every fault of the four families is surfaced. Nondeterminism is handled by seeding the server and by repeating runs; identical seeds are not guaranteed to reproduce identical text across hardware and server versions [38]. Every number in the paper is generated by one script from the released logs.

V. RESULTS

A. Sanity replication of the earlier evaluation

As a check that the substrate behaves as before on more repositories, we re-ran the earlier evaluation (Table VI). On **XX** repositories, the transformation-based hand-offs kept **XX** of **XX** canaries with the 7B model against **XX** for free text and **XX** for shared schema, and change impact was **XX/XX** (precision/recall) at **XX**k tokens per change. This is a replication of previous work, not a result of this paper, and anchors the setting for what follows.

B. RQ1: Delivery and traceability of knowledge

Delivery. After a revision, **XX** of the artifacts that had to change carried the new marker under ctxi (routed: **XX**; paste: **XX**; ctx: **XX**), and **XX** still carried the old one (routed: **XX**; paste: **XX**) (Table VII). Delivery separates two failures: the runtime not re-running an artifact, and a small model not writing the marker when it does. Regeneration churn per step was **XX** (ctxi), **XX** (ctx), **XX** (routed), and **XX** (paste).

TABLE VIII

RQ2. MEAN TOKENS (THOUSANDS) AND LLM CALLS PER STEP, OVER REPOSITORIES, SEEDS AND MODELS. R0 IS THE BUILD; R1–R5 ARE THE REVISIONS OF SECTION IV.

Step	Paste tok	calls	Routed tok	calls	Ctx tok	calls	Ctx-item tok	calls
R0 build	XX	XX	XX	XX	XX	XX	XX	XX
R1 revise test item	XX	XX	XX	XX	XX	XX	XX	XX
R2 revise code item	XX	XX	XX	XX	XX	XX	XX	XX
R3 add unrelated item	XX	XX	XX	XX	XX	XX	XX	XX
R4 identical rewrite	XX	XX	XX	XX	XX	XX	XX	XX
R5 revise code item	XX	XX	XX	XX	XX	XX	XX	XX

[figure context_cost – placeholder]

Fig. 2. Cumulative cost of absorbing the five revisions (left) and impact precision on steps with a non-empty ground truth (right), by policy.

Implementation check of Property 1. With item-level pins the runtime marked exactly the ground-truth artifacts in **XX** of the **XX** steps with a non-empty ground truth (precision **XX**, recall **XX**); on the **XX** steps with an empty one, **XX** re-ran anything, against **XX** for whole-context pins (an unrelated item invalidates every consumer); a step counts as a re-run only if it made LLM calls. Departures from exactness have one cause: a binding that failed validation on an earlier revision keeps its old stamp, so it stays an obligation (lowering precision) but its re-run is cached as escalated and costs nothing, and a binding that never obtained a value holds no stamp (lowering recall) and is retried on every run. On steps with no failed bindings the marked set was exact in **XX** of **XX** steps. Naive paste’s precision of **XX** and routed paste’s of **XX** are properties of the policies, not findings.

Influence. *Infl* computed from pins agreed with the items present in the prompts with precision **XX** and recall **XX** (item-level) and **XX/XX** (whole-context). The paste policies have no pin to query: the finding text sits inside a rule’s prompt literal. A text search over rule files can recover the same set for a team that knows to look, which is the manual process the query replaces. This check verifies the implementation of Section III; it does not show that every influence is detected.

C. RQ2: Cost of distributing knowledge

Cost. Absorbing the five revisions cost **XX**k tokens and **XX** calls per run with item-level pins, against **XX**k and **XX** for routed paste, **XX**k and **XX** for naive paste, and **XX**k and **XX** for whole-context pins (Table VIII, Fig. 2). Summed over repository–model pairs, naive paste spent **XX**×, whole-context pins **XX**×, and routed paste **XX**× the tokens of ctxi (in calls: ×**XX**, ×**XX**, ×**XX**). The paired 95% interval of the per-pair saving against naive paste is [**XX**, **XX**]k tokens ($p=\mathbf{XX}$, $n=\mathbf{XX}$), and against routed paste [**XX**, **XX**]k tokens ($p=\mathbf{XX}$). With two findings and two consumer kinds the saving against

TABLE IX

RQ3. INJECTED FAULTS: SHARE SURFACED BY THE INTENDED SIGNAL, AND SHARE THAT CORRUPTED STATE OR ENDED WITH THE RUN REPORTING ACCEPTANCE (SILENT). THE TWO DESIGN-LIMIT ROWS ARE PROBES OF WHAT THE DESIGN CANNOT SEE. COUNTS n ARE REPOSITORIES $\times$ SEEDS $\times$ MODELS AND ARE NOT INDEPENDENT.

Injected fault	Family	n	Surfaced	Silent
reader access revoked	context access	XX	XX	XX
context expired / missing	context access	XX	XX	XX
write by non-writer, forged author	context access	XX	XX	XX
concurrent writers	context access	XX	XX	XX
LLM output rejected / transport failure	llm	XX	XX	XX
bad signature, wrong key, unknown agent	nostr integrity	XX	XX	XX
wrong namespace, tampered, forked base	nostr integrity	XX	XX	XX
relay offline	nostr availability	XX	XX	XX
unrelated context changed (control)	benign	XX	XX	XX
rule prompt edited (undeclared dependency)	design limit	XX	XX	XX
wrong knowledge from an authorized writer	design limit	XX	XX	XX

naive paste is bounded by a small factor set by the schedule, so the informative comparison is routed paste: selective pinning is expected to match its cost, and what it removes is the hand-maintained table, not calls. The build costs more with a context than with pasted text (XXk and XXk for ctx and ctxi against XXk for paste) because each prompt carries a labelled section.

Fidelity guard. Canary retention after the last revision was XX (ctxi), XX (routed), XX (ctx), and XX (paste), against XX, XX, XX, and XX after the build. The per-repository difference between ctxi and routed paste is XX (95% interval [XX, XX], non-inferior within one canary of five: XX), and against naive paste XX ([XX, XX], non-inferior: XX). As in the earlier evaluation, generated modules rarely pass a repository’s full suite with these models, so the guard supports equal retention of stated constants and not equal correctness.

Timeline metrics. The timeline reproduced the number of LLM calls and tokens exactly in XX of runs, with server-reported token counts (transport failures carry none), so cost can be recomputed from events alone. Per run, the context-reuse metric was XX accepted bindings and context-induced obligations were XX (ctxi) against XX (ctx).

D. RQ3: Invariants and failure surfacing

Non-interference. Across XX replayed runs, attaching Nostr left the final state identical in XX and the prompt multiset identical in XX; with the relay down, XX and XX. The replay reproduced the original run’s state in XX, and XX prompts were absent from the recordings. This supports Property 3 for deterministic replays; it does not exercise network timing or concurrency. A relay outage queued XX events on average, XX of the outboxes drained completely on flush, and peers converged on the same context digest in XX.

Reconstruction. From the timeline alone, XX of accepted values had their footprint version and pinned context versions recovered; from the verified Nostr events alone, XX. XX of published events verified. As stated in Section IV, this shows the event schema is sufficient, not that it is independent evidence.

Overhead. With the LLM answered instantly, the timeline added XX ms per event (XX of engine time), and signing and

queueing for Nostr added XX ms per event. A run produced XX events (XX kB). This is the worst case for relative overhead; against LLM calls of seconds it is not measurable, but it is reported so the instant-backend ratio is not hidden.

Privacy. Standard-mode timeline and Nostr timeline events contained XX occurrences of markers, generated code, or prompt text. Snapshot events for `visibility: relay` contexts carry content by design (XX observed). This tests our chosen markers; it does not rule out inference from digests, target keys, or error messages.

Faults. Of XX injections in the four families, XX were surfaced by the intended signal and XX ended silently, and the benign control raised no obligation in XX of cases (Table IX). When a reader’s access was revoked, a context expired, or a context was missing, the first error event came XX events into the run on median, and XX prompts were sent without the context they declared. The two design-limit probes were surfaced in XX of cases and ended silently in XX: an edit to a rule’s prompt text creates no obligation, and the runtime treats a contradictory finding from an authorized writer as an ordinary revision.

VI. DISCUSSION

What the mechanisms buy. Against naive paste the saving comes from not re-running consumers that do not depend on a change. Against routed paste the cost should be close, and the difference the design makes is what maintains the routing: the stamp recomputes what depends on what, the pins record it, and the timeline shows it. This is a claim about automation and auditability, and the study measures it only through what the team does *not* have to maintain. Whether a hand-kept table drifts is a human-factors question that this study does not test.

Where savings scale. A revision of item j costs $\propto N_j$ consumers under ctxi and routed paste, and $\propto \sum_j N_j$ under naive paste and ctx, where N_j is the number of operations consuming item j (here 4–17 per repository, the same for both findings). The gap to naive paste and ctx grows with the number of independently consumed findings, which is two here; the per-prompt overhead of a context section is paid once per generation. Team-size scaling is where the argument has most leverage and where this study has least data.

The limits are part of the result. Pinning is only as good as the declaration: a rule author who pastes knowledge into a prompt literal, or edits a rule, gets no obligations (the probe above), and nothing in the design judges whether a finding is right. The fail-closed property turns a missing or revoked context into a blocked binding, which is safe and also a liveness cost: one expired context blocks all its dependents.

Implications. Declare which findings a role consumes and let the stamp decide what to regenerate; keep contexts team-local unless they must cross machines; treat blocked bindings as signals. For researchers, knowledge traceability for LLM teams becomes an empirical question, and fault injection against a coordination runtime, with probes of what the design cannot see, is a reusable method.

VII. THREATS TO VALIDITY

Construct. (i) Marker presence measures whether a model followed a literal instruction, not whether knowledge improved the code. The findings are synthetic so ground truth is decidable; a scenario in which knowledge changes behavior (a required call, a forbidden pattern checked by static analysis or tests) and where stale knowledge causes a defect would be more convincing and is future work. (ii) Naive paste is the practice the mechanisms replace but is a weak baseline; routed paste is the strong manual one, with a routing table assumed correct. (iii) Impact precision, spurious re-runs, and fault surfacing are partly fixed by definitions and are reported as checks. Influence precision compares two views of the same pins and prompts. (iv) Reconstruction accuracy is circular because the engine emits the events; it shows sufficiency of the schema only.

Internal. Sampling is nondeterministic. We seed the server and repeat runs, but identical seeds are not guaranteed to reproduce identical text across hardware and server versions. Regeneration churn combines sampling noise with real effects. The Nostr control is a replay of recorded answers, which isolates the projection from sampling but not from network timing; the relay is in memory, and latency, partial delivery, and hostile relays are modeled by injection. The study’s harness changed while runs were in progress (a routed-paste arm and two probes were added after review); seeds produced before the change lack those arms and the table counts show which runs contain them.

External. **XX** Python repositories, two open-weight model families, one team topology, one scenario family, and two findings: the reported savings describe this regime and are a lower bound for teams with many findings. We make no claims for closed models, other languages, or human-agent teams.

Conclusion. Units are repositories per model; the number of repositories bounds what a test can establish, so per-repository values and intervals are primary. Effect directions were fixed before the final runs but were not registered externally.

VIII. RELATED WORK

Incremental computation and transformation. Build systems and demand-driven computation re-run only what a changed input affects [27], [32], [39], and incremental model transformation and queries do the same for models [40]–[42]. Hashing a declared read set into a cache key is standard in this literature, and Property 1 is that idea. What is new is the setting and the combination: the leaf is a stochastic function, the input is *knowledge* that is not an artifact of any view, and the input is versioned, access-controlled, attributed, and synchronizable, with a fail-closed rule and an influence query over what was used.

Traceability and provenance. Software traceability links artifacts across lifecycle stages for impact analysis [43], [44], and learning-based approaches recover links after the fact [45]. We record links as a by-product of transformation and extend them with knowledge pins, so influence is read from declared

dependencies rather than recovered. Trace models persist transformation links [30], [31]; provenance in databases and workflows [46], [47] is the closest conceptual relative, and our pins are a coarse declared provenance limited to knowledge.

Shared knowledge for agents. Blackboard systems [33], [48] let specialist knowledge sources cooperate through a shared store, and LLM memory systems [34], [49] manage what an agent remembers. Role-based and conversation-based frameworks [2], [3], [25], [50], [51] share state through messages, and MCP exposes shared resources to a model [22]. The addition here is a contract (access policy, versioning, a declared read set) that feeds an invalidation rule and a query.

Context cost. Prompt compression [52] reduces tokens per call, long-context studies show degradation with length and position [20], and provider-side prefix caching reduces the cost of re-sending an unchanged prefix. Selective pinning reduces how often an input must be regenerated at all and composes with both; we do not compare against caching, which changes price and latency, not what is re-run.

Observability and protocols. Agent-ops work defines trace and artifact taxonomies for LLM agents [16], and telemetry standards define spans and metrics [53], including conventions for generative-AI calls. Interoperability protocols standardize transport [22]–[24]. Our plane is a *projection* designed not to influence the run, with a verified signed transport, and records knowledge versions rather than only calls. Replicated data types [37] are the usual route to cross-machine sharing; we use a single-writer, version-continuity discipline that the local policy can audit and give up convergence with several writers, ordering by version numbers rather than clocks [54].

Agent benchmarks. SWE-bench [55] and agent-computer interfaces [56] evaluate single agents on issues; DevBench [28] evaluates the development lifecycle. We reuse DevBench for its real tests and add canaries and knowledge markers that isolate hand-off and knowledge fidelity from task difficulty.

IX. CONCLUSION

Shared knowledge in an agent team can be a first-class, versioned, access-controlled object whose selected item digest joins the binding stamp. Under that design, invalidation is as fine as the declared selection, influence is a query, unresolvable context fails closed, and, under stated assumptions, observing a run does not change it. On **XX** repositories, item-level pinning delivered revised knowledge to **XX** of the artifacts that had to change, cost **XX**× the tokens of hand-routed paste without a hand-maintained table, and left state and prompts identical when a relay was attached or down. Two probes show the boundary: edits to a prompt literal and wrong-but-authorized knowledge are invisible to the design. Future work includes scenarios where knowledge changes behavior and stale knowledge causes defects, many findings and consumers, human-maintained routing tables as a baseline, wide-area relays, encrypted context transport, and flows in which a downstream agent corrects shared knowledge.

Data availability. The runtime, harness, reference-validity script, raw logs, generated tables and figures, the aggregation

script that produces every number quoted here, and the environment record (server and model versions, hardware) are available at an anonymized repository (<https://anonymous.4open.science/r/context-pinning-study>) and will be archived with a persistent identifier upon acceptance. The artifact README lists the commands that regenerate each table and a mock-backend smoke test.

REFERENCES

- [1] Y. Dong, X. Jiang, Z. Jin, and G. Li, “Self-collaboration code generation via ChatGPT,” *ACM Trans. Softw. Eng. Methodol.*, vol. 33, no. 7, pp. 1–38, 2024.
- [2] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, C. Zhang, J. Wang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber, “MetaGPT: Meta programming for a multi-agent collaborative framework,” in *Proc. ICLR*, 2024.
- [3] C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, C. Yang, W. Chen, Y. Su, X. Cong, J. Xu, D. Li, Z. Liu, and M. Sun, “ChatDev: Communicative agents for software development,” in *Proc. ACL*, 2024.
- [4] W. Tao, Y. Zhou, Y. Wang, W. Zhang, H. Zhang, and Y. Cheng, “MAGIS: LLM-based multi-agent framework for GitHub issue resolution,” in *Proc. NeurIPS*, 2024.
- [5] J. He, C. Treude, and D. Lo, “LLM-based multi-agent systems for software engineering: Literature review, vision, and the road ahead,” *ACM Trans. Softw. Eng. Methodol.*, vol. 34, no. 5, 2025.
- [6] M. Cemri, M. Z. Pan, S. Yang, L. A. Agrawal, B. Chopra, R. Tiwari, K. Keutzer, A. Parameswaran, D. Klein, K. Ramchandran, M. Zaharia, J. E. Gonzalez, and I. Stoica, “Why do multi-agent LLM systems fail?” in *Proc. NeurIPS, Datasets and Benchmarks Track*, 2025.
- [7] A. J. Ko, R. DeLine, and G. Venolia, “Information needs in collocated software development teams,” in *Proc. 29th Int. Conf. Softw. Eng. (ICSE)*, 2007, pp. 344–353.
- [8] C. R. B. de Souza and D. F. Redmiles, “An empirical study of software developers’ management of dependencies and changes,” in *Proc. 30th Int. Conf. Softw. Eng. (ICSE)*, 2008, pp. 241–250.
- [9] M. Cataldo and J. D. Herbsleb, “Coordination breakdowns and their impact on development productivity and software failures,” *IEEE Trans. Softw. Eng.*, vol. 39, no. 3, pp. 343–360, 2013.
- [10] J. D. Herbsleb and A. Mockus, “An empirical study of speed and communication in globally distributed software development,” *IEEE Trans. Softw. Eng.*, vol. 29, no. 6, pp. 481–494, 2003.
- [11] A. Mockus and J. D. Herbsleb, “Expertise browser: A quantitative approach to identifying expertise,” in *Proc. 24th Int. Conf. Softw. Eng. (ICSE)*, 2002, pp. 503–512.
- [12] J. T. Liang, M. Lin, N. Rao, and B. A. Myers, “Prompts are programs too! Understanding how developers build software containing prompts,” *Proc. ACM Softw. Eng.*, vol. 2, no. FSE, pp. 1591–1614, 2025.
- [13] C. Qian, Y. Dang, J. Li, W. Liu, Z. Xie, Y. Wang, W. Chen, C. Yang, X. Cong, X. Che, Z. Liu, and M. Sun, “Experiential co-learning of software-developing agents,” in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics (ACL)*, 2024, pp. 5628–5640.
- [14] R. Bairi, A. Sonwane, A. Kanade, V. D. C., A. Iyer, S. Parthasarathy, S. Rajamani, B. Ashok, and S. Shet, “CodePlan: Repository-level coding using LLMs and planning,” *Proc. ACM Softw. Eng.*, vol. 1, no. FSE, pp. 675–698, 2024.
- [15] I. Bouzenia and M. Pradel, “Understanding software engineering agents: A study of thought-action-result trajectories,” in *Proc. 40th IEEE/ACM Int. Conf. Autom. Softw. Eng. (ASE)*, 2025, pp. 2846–2857.
- [16] L. Dong, Q. Lu, and L. Zhu, “AgentOps: Enabling observability of LLM agents,” arXiv:2411.05285, 2024.
- [17] Z. Zhang, C. Wang, Y. Wang, E. Shi, Y. Ma, W. Zhong, J. Chen, M. Mao, and Z. Zheng, “LLM hallucinations in practical code generation: Phenomena, mechanism, and mitigation,” *Proc. ACM Softw. Eng.*, vol. 2, no. ISSTA, pp. 481–503, 2025.
- [18] M. Rath, J. Rendall, J. L. C. Guo, J. Cleland-Huang, and P. Mäder, “Traceability in the wild: Automatically augmenting incomplete trace links,” in *Proc. 40th Int. Conf. Softw. Eng. (ICSE)*, 2018, pp. 834–845.
- [19] X. Chen, C. Gao, C. Chen, G. Zhang, and Y. Liu, “An empirical study on challenges for LLM application developers,” *ACM Trans. Softw. Eng. Methodol.*, vol. 34, no. 7, pp. 1–37, 2025.
- [20] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, “Lost in the middle: How language models use long contexts,” *Trans. Association for Computational Linguistics*, vol. 12, 2024.
- [21] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang, “Demystifying LLM-based software engineering agents,” *Proc. ACM Softw. Eng.*, vol. 2, no. FSE, pp. 801–824, 2025.
- [22] Anthropic, “Model context protocol,” Specification, <https://modelcontextprotocol.io>, 2024.
- [23] Google, “Agent2Agent (A2A) protocol,” Specification, <https://a2a-protocol.org>, 2025.
- [24] A. Ehteshami, A. Singh, G. K. Gupta, and S. Kumar, “A survey of agent interoperability protocols: MCP, ACP, A2A, and ANP,” arXiv:2505.02279, 2025.
- [25] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. Awadallah, R. W. White, D. Burger, and C. Wang, “AutoGen: Enabling next-gen LLM applications via multi-agent conversation,” in *Proc. COLM*, 2024.
- [26] Anonymous, “AgentM2M: M2M transformations for agentic collaborations in software engineering,” in *Proc. ICSE New Ideas and Emerging Results (NIER)*, under review, 2027.
- [27] A. Mokhov, N. Mitchell, and S. Peyton Jones, “Build systems à la carte,” *Proc. ACM Program. Lang. (ICFP)*, vol. 2, 2018.
- [28] B. Li, W. Wu, Z. Tang, L. Shi, J. Yang, J. Li, S. Yao, C. Qian, B. Hui, Q. Zhang, Z. Yu, H. Du, P. Yang, D. Lin, C. Peng, and K. Chen, “DevBench: A comprehensive benchmark for software development,” arXiv:2403.08604, 2024.
- [29] F. Jouault, F. Allilaire, J. Bézivin, and I. Kurtev, “ATL: A model transformation tool,” *Sci. Comput. Program.*, vol. 72, no. 1–2, pp. 31–39, 2008.
- [30] F. Jouault, “Loosely coupled traceability for ATL,” in *Proc. ECMDA Workshop on Traceability*, 2005.
- [31] J. M. Vara, V. A. Bollati, Á. Jiménez, and E. Marcos, “Dealing with traceability in the MDD of model transformations,” *IEEE Trans. Softw. Eng.*, vol. 40, no. 6, pp. 555–583, 2014.
- [32] M. A. Hammer, K. Y. Phang, M. Hicks, and J. S. Foster, “Adapton: Composable, demand-driven incremental computation,” in *Proc. ACM PLDI*, 2014.
- [33] H. P. Nii, “Blackboard systems: The blackboard model of problem solving and the evolution of blackboard architectures,” *AI Magazine*, vol. 7, no. 2, pp. 38–53, 1986.
- [34] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez, “MemGPT: Towards LLMs as operating systems,” arXiv:2310.08560, 2023.
- [35] Nostr contributors, “NIP-01: Basic protocol flow description,” Nostr Implementation Possibilities, 2023.
- [36] P. Wuille, J. Nick, and T. Ruffing, “BIP 340: Schnorr signatures for secp256k1,” Bitcoin Improvement Proposal, 2020.
- [37] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski, “Conflict-free replicated data types,” in *Proc. Int. Symp. Stabilization, Safety, and Security of Distributed Systems (SSS)*, 2011.
- [38] A. Arcuri and L. Briand, “A practical guide for using statistical tests to assess randomized algorithms in software engineering,” in *Proc. ICSE*, 2011, pp. 1–10.
- [39] N. Mitchell, “Shake before building: Replacing make with Haskell,” in *Proc. ACM ICFP*, 2012.
- [40] F. Jouault and M. Tisi, “Towards incremental execution of ATL transformations,” in *Proc. ICMT*, ser. LNCS, vol. 6142, 2010, pp. 123–137.
- [41] Z. Ujhelyi, G. Bergmann, Á. Hegedüs, Á. Horváth, B. Izsó, I. Ráth, Z. Szatmári, and D. Varró, “EMF-IncQuery: An integrated development environment for live model queries,” *Science of Computer Programming*, vol. 98, pp. 80–99, 2015.
- [42] H. Giese and R. Wagner, “From model transformation to incremental bidirectional model synchronization,” *Software & Systems Modeling*, vol. 8, no. 1, pp. 21–43, 2009.
- [43] O. C. Z. Gotel and A. C. W. Finkelstein, “An analysis of the requirements traceability problem,” in *Proc. IEEE Int. Conf. Requirements Engineering (ICRE)*, 1994, pp. 94–101.
- [44] J. Cleland-Huang, O. C. Z. Gotel, J. Huffman Hayes, P. Mäder, and A. Zisman, “Software traceability: Trends and future directions,” in *Future of Software Engineering (FOSE) at ICSE*, 2014, pp. 55–69.
- [45] J. Guo, J. Cheng, and J. Cleland-Huang, “Semantically enhanced software traceability using deep learning techniques,” in *Proc. ICSE*, 2017, pp. 3–14.

- [46] P. Buneman, S. Khanna, and W.-C. Tan, "Why and where: A characterization of data provenance," in *Proc. Int. Conf. Database Theory (ICDT)*, 2001.
- [47] L. Moreau and P. Missier, "PROV-DM: The PROV data model," W3C Recommendation, 2013.
- [48] L. D. Erman, F. Hayes-Roth, V. R. Lesser, and D. R. Reddy, "The Hearsay-II speech-understanding system: Integrating knowledge to resolve uncertainty," *ACM Computing Surveys*, vol. 12, no. 2, pp. 213–253, 1980.
- [49] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative agents: Interactive simulacra of human behavior," in *Proc. ACM UIST*, 2023.
- [50] G. Li, H. A. A. K. Hammoud, H. Itani, D. Khizbullin, and B. Ghanem, "CAMEL: Communicative agents for "mind" exploration of large language model society," in *Proc. NeurIPS*, 2023.
- [51] W. Chen, Y. Su, J. Zuo, C. Yang, C. Yuan, C.-M. Chan, H. Yu, Y. Lu, Y.-H. Hung, C. Qian, Y. Qin, X. Cong, R. Xie, Z. Liu, M. Sun, and J. Zhou, "AgentVerse: Facilitating multi-agent collaboration and exploring emergent behaviors," in *Proc. ICLR*, 2024.
- [52] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, and L. Qiu, "LLMLingua: Compressing prompts for accelerated inference of large language models," in *Proc. EMNLP*, 2023.
- [53] Cloud Native Computing Foundation, "OpenTelemetry specification," <https://opentelemetry.io>, 2024.
- [54] L. Lamport, "Time, clocks, and the ordering of events in a distributed system," *Commun. ACM*, vol. 21, no. 7, pp. 558–565, 1978.
- [55] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?" in *Proc. ICLR*, 2024.
- [56] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "SWE-agent: Agent-computer interfaces enable automated software engineering," in *Proc. NeurIPS*, 2024.